Case Study: Online Course Management System

Scenario

A startup is building an **Online Course Management System** where instructors can create courses and upload course materials, while students can enroll in courses and access content.

You are asked to build a **Spring Boot REST API** that manages users, courses, enrollments, and course materials.

The system must support:

- User registration and login
- Course creation and management
- Student enrollment
- Uploading course materials
- Viewing courses with pagination
- Proper exception handling and validation
- API documentation using Swagger

Security is **optional**. If time permits after completing the core application, you may add **Spring Security with JWT**.

Functional Requirements

The system should support the following modules:

1. User Management
2. Course Management
3. Enrollment Management
4. Course Material Management
5. File Upload & Download
6. Pagination and Sorting
7. Validation & Exception Handling
8. API Documentation (Swagger)
9. Caching

Step 1 — Create Spring Boot Project

Use **Spring Initializr**.

Project Configuration

Project: Maven
Language: Java
Spring Boot: Latest stable version
Group: com.learning
Artifact: course-management-system
Packaging: Jar
Java: 17 or above

Step 2 — Add Required Dependencies

Students should include the following dependencies:

- Spring Web
- Spring Data JPA
- PostgreSQL Driver
- Validation
- Lombok
- Spring Boot DevTools
- SpringDoc OpenAPI (Swagger)
- Spring Cache
- Spring Boot Starter AOP

(Optional)

- Spring Security
- JWT libraries

Step 3 — Database Configuration

Use **PostgreSQL**.

Students must configure:

application.properties

Include:

- database connection
- Hibernate configuration
- file upload configuration
- cache configuration

Step 4 — Suggested Project Structure

Students must organize the project using **layered architecture**.

com.learning.cms

config
controllers
dto
entity
exception
mapper
repository
service
service.impl
util

Explanation:

Package	Purpose
config	Swagger, Caching, CORS configuration
controllers	REST Controllers

dto	Request and Response DTOs
entity	Database entities
exception	Custom exceptions
mapper	DTO ↔ Entity mapping
repository	JPA repositories
service	Business logic interfaces
service.impl	Service implementations
util	File utilities, constants

Step 5 — Entity Design

Students must create the following entities with proper **relationships**.

---

1 User Entity

Represents system users.

Fields

id
fullName
email
password
role
profilePicture
createdAt
updatedAt

Enum

Role
ADMIN
INSTRUCTOR
STUDENT

Relationships

User → Course

One Instructor can create many Courses
OneToMany (User → Course)

User → Enrollment

One Student can enroll in many Courses
OneToMany (User → Enrollment)

---

2 Course Entity

Represents courses created by instructors.

Fields

id
title
description
price
duration
level
createdAt
updatedAt

Relationships

Course → Instructor

ManyToOne
Course belongs to one Instructor

Course → Enrollment

OneToMany
Course has many enrollments

Course → CourseMaterial

OneToMany
Course contains multiple materials

---

3 Enrollment Entity

Represents a student enrolling in a course.

Fields

id
enrollmentDate
status
progressPercentage

Enum

EnrollmentStatus
ACTIVE
COMPLETED
CANCELLED

Relationships

ManyToOne → User (Student)
ManyToOne → Course

This entity resolves the **Many-to-Many relationship between Student and Course**.

---

4 CourseMaterial Entity

Represents files uploaded for a course.

Fields

id
title
fileName
fileType
fileUrl
uploadDate

Relationship

ManyToOne → Course

One course can have **multiple materials**.

Step 6 — DTO Design

Students must **not expose entities directly**.

Create DTOs.

User DTOs

RegisterRequestDTO

Fields

fullName
email
password
role

LoginRequestDTO

email
password

UserResponseDTO

id
fullName
email
role
profilePicture
createdAt

Course DTOs

CourseRequestDTO

title
description
price
duration
level

CourseResponseDTO

id
title
description
price
duration
level
instructorName
createdAt

Enrollment DTOs

EnrollmentRequestDTO

courseId
studentId

EnrollmentResponseDTO

id
courseTitle
studentName
status
progressPercentage
enrollmentDate

Course Material DTOs

MaterialUploadDTO

title
courseId
file

MaterialResponseDTO

id
title
fileName
fileType
fileUrl
uploadDate

Step 7 — Repository Layer

Students must create repositories extending:

JpaRepository

Repositories required:

UserRepository
CourseRepository
EnrollmentRepository
CourseMaterialRepository

Custom methods can include:

findByEmail
findByInstructorId
findByCourseId
findByStudentId

Step 8 — Service Layer

Service interfaces should define business operations.

Example modules:

UserService

Responsibilities

- register user
- login user
- fetch user profile

CourseService

Responsibilities

- create course
- update course
- delete course
- list courses (pagination)
- get course by id

EnrollmentService

Responsibilities

- enroll student
- view student enrollments
- update progress

CourseMaterialService

Responsibilities

- upload material
- download material
- list course materials

Step 9 — Controller Layer

Students must implement REST endpoints.

User APIs

POST /api/auth/register
POST /api/auth/login
GET /api/users/{id}

Course APIs

POST   /api/courses
PUT   /api/courses/{id}
DELETE /api/courses/{id}

GET /api/courses
GET /api/courses/{id}

Pagination example:

GET /api/courses?page=0&size=10&sort=title

Enrollment APIs

POST /api/enrollments
GET /api/enrollments/student/{studentId}
GET /api/enrollments/course/{courseId}

Course Material APIs

POST /api/materials/upload
GET /api/materials/{id}/download
GET /api/materials/course/{courseId}

Step 10 — Validation

Students must apply **Bean Validation**.

Examples:

@NotBlank
@Email
@Size
@Positive

Validate DTO fields such as:

- email
- password
- course title

- price

Step 11 — Exception Handling

Students must implement **Global Exception Handling**.

Create:

GlobalExceptionHandler

Handle exceptions such as:

- ResourceNotFoundException
- InvalidRequestException
- FileStorageException
- MethodArgumentNotValidException

Use:

@ControllerAdvice

Step 12 — Pagination and Sorting

Students must implement pagination using:

Pageable
PageRequest
Sort

Example API:

GET /api/courses?page=0&size=5&sort=title,asc

Step 13 — File Upload & Download

Students must implement:

Upload

MultipartFile

Store files in:

uploads/

Download endpoint should return:

ResponseEntity<Resource>

Step 14 — Swagger Documentation

Students must integrate **OpenAPI / Swagger UI**.

Swagger should document:

- API endpoints
- request bodies
- response models

Access Swagger UI:

http://localhost:8080/swagger-ui/index.html

Step 15 — Caching

Students must implement caching for **frequently accessed APIs**.

Example:

GET /api/courses

Use:

@Cacheable
@CacheEvict

Optional Challenge (Advanced)

If time permits after completing the project:

Add **Spring Security with JWT authentication**.

Features:

- login authentication
- role based access
- protected APIs

Example roles:

ADMIN
INSTRUCTOR
STUDENT